

13. Unicode aur String Internals

| Advanced topic: emoji, rare characters ke saath kaam karna ho tab kaam aata hai.

Character likhne ke 3 tarike

Notation	Matlab
<code>\xXX</code>	2 hex digits, sirf pehle 256 characters
<code>\uXXXX</code>	4 hex digits
<code>\u{X...XXXXXX}</code>	1 se 6 hex digits, sab Unicode characters

```
js
"\x7A"      // z
"\u00A9"    // ©
"\u{1F60D}" // 🍌
```

Surrogate Pairs

- JS pehle sirf 2-byte (UTF-16) characters sochta tha, jisme 65536 hi combinations hain.
- Rare symbols (emoji, rare Chinese chars) **2 characters ki jodi** (surrogate pair) me store hote hain.
- Isliye:

```
js
'🍌'.length; // 2 (dikhta ek hai, length 2)
'x'[0];      // kachra (pair ka aadha hissa)
```

- Pair ka pehla hissa `0xD800..0xDBFF` me hota hai, doosra `0xDC00..0xDFFF` me.

Sahi methods:

```
js
'x'.charCodeAt(0); // galat, sirf pehla hissa
'x'.codePointAt(0); // sahi, poora code
String.fromCodePoint(...) // sahi
```

Khatra: string ko kahin bhi kaatna (`slice`) unsafe hai:

```
js
```

```
'hi 😊'.slice(0, 4); // "hi " + adhoora emoji (kachra)
```

Diacritical marks aur Normalization

- Kai characters = base letter + mark (jaise `Š` + dot above `\u0307`).
- Do strings **dikhne me same** ho sakti hain par unki Unicode composition alag ho sakti hai:

```
js
```

```
let s1 = 'Š\u0307\u0323';  
let s2 = 'Š\u0323\u0307';  
s1 == s2; // false, jabki dono 'dikhate' hain
```

- Fix: `normalize()` dono ko ek standard form me la deta hai:

```
js
```

```
s1.normalize() == s2.normalize(); // true
```

Yaad rakho

- `length` emoji par bharosemand nahi.
- Code point ke liye `codePointAt` / `fromCodePoint` use karo.
- String compare karte waqt `normalize()` bhi socho.